

Atomic ինչ է — Խորը ուղեցույց

C++ Concurrency presentation

Այս ուղեցույցը մանրամասն բացատրում է `std::atomic`-ը՝ ինչ է, ինչպես է լուծում race-ը առանց mutex-ի, atomic vs mutex, operation-ներ, memory order, օրինակներով և հարցազրույցի հարցերով:

1. Ի՞նչ է Atomic-ը

`std::atomic`-ը տեսակ է, որի operation-ները **անբաժանելի (atomic)** են -- մեկ անբաժանելի քայլ, որի մեջ այլ thread չի կարող միջամտել: Չկա race condition առանց mutex-ի:

```
#include <atomic>
std::atomic<int> counter{0};

void increment() {
    counter++;    // ATOMIC -- read-modify-write մեկ անբաժանելի քայլ
}
// 2 thread -> counter ճիշտ է, առանց mutex-ի
```

2. Խնդիրը — ինչու սովորական int-ը race է

```
int counter = 0;    // սովորական
counter++;           // 3 քայլ: read, +1, write -> thread-ներ խառնվում
```

```
std::atomic<int> counter{0};    // atomic
counter++;                       // Մեկ անբաժանելի քայլ -> չկա race
```

Atomic-ը երաշխավորում է, որ `counter++`-ը կատարվի ամբողջովին կամ ոչինչ (չկա միջանկյալ վիճակ):

3. Atomic operation-ներ

```
std::atomic<int> a{0};

a++; ++a;           // atomic increment
a--; --a;           // atomic decrement
a += 5;             // atomic add
a.load();           // atomic read
a.store(10);         // atomic write
a.exchange(20);      // set նոր, վերադարձնում հին
int expected = 20;
a.compare_exchange_strong(expected, 30); // CAS (compare-and-swap)
```

compare_exchange (CAS) — lock-free-ի հիմք

```
// եթե a == expected -> a = desired, return true
// հակառակ դեպքում -> expected = a, return false
std::atomic<int> a{5};
int expected = 5;
a.compare_exchange_strong(expected, 10); // a դարձավ 10
```

CAS-ը lock-free ալգորիթմների հիմքն է:

4. Atomic vs Mutex

ATOMIC:

- + Ավելի արագ (lock-free, չկա blocking)
- + Փոքր operation-ի համար (counter, flag, pointer)
- Միայն ՊԱՐՉ operation (մեկ փոփոխական)
- Բարդ logic-ի համար չի աշխատում

MUTEX:

- + Բարդ critical section (մի քանի փոփոխական, բարդ logic)
- Ավելի դանդաղ (blocking, context switch)

	atomic	mutex
Արագություն	արագ (lock-free)	դանդաղ (blocking)
Կիրառ	փոքր operation (counter/flag)	բարդ critical section
Blocking	ոչ	այո
Բարդ logic	ոչ	այո

Կանոն. փոքր (counter, flag, pointer) -> atomic; բարդ (մի քանի փոփոխական, container փոխում) -> mutex:

5. Իրական օրինակ

Atomic counter

```
std::atomic<int> counter{0};
void work() {
    for (int i = 0; i < 1000000; i++)
        counter++;    // atomic, առանց mutex, արագ
}
```

Atomic flag (stop signal)

```
std::atomic<bool> stop{false};

void worker() {
    while (!stop.load())    // atomic read
        doWork();
}

void main_control() {
    stop.store(true);        // atomic write -> worker կանգնում
}
```

6. Memory Order (առաջացված)

```
a.store(10, std::memory_order_relaxed); // ոչ մի կարգ երաշխիք (արագ)
a.load(std::memory_order_acquire);     // acquire
a.store(10, std::memory_order_release); // release
// default: memory_order_seq_cst (ամենախիստ, ամենաանվտանգ)
```

Junior մակ. default (`seq_cst`) բավական է; memory order-ը optimization է առաջացված դեպքերում:

7. is_lock_free

```
std::atomic<int> a;
cout << a.is_lock_free(); // սովորաբար true փոքր տիպերի համար
```

Փոքր տիպեր (int, pointer) սովորաբար lock-free են (ուղիղ CPU instruction); մեծ object-ներ կարող են ներսում mutex օգտագործել:

8. Հարցազրույցի հարցեր և պատասխաններ

atomic ինչ է:

Տեսակ, որի operation-ները անբաժանելի են; race condition առանց mutex-ի փոքր operation-ների համար:

Ինչո՞ւ սովորական int-ը race է, atomic-ը ոչ:

int++ 3 քայլ է (read/+1/write); atomic++-ը մեկ անբաժանելի քայլ:

atomic vs mutex:

atomic արագ (lock-free) փոքր operation-ի; mutex բարդ critical section-ի (blocking). Փոքր -> atomic, բարդ -> mutex:

compare_exchange (CAS) ինչ է:

Եթե արժեքը == expected, փոխում desired-ով (atomic); lock-free ալգորիթմների հիմք:

Memory order ինչ է:

Atomic operation-ների կարգավորման երաշխիքներ; default seq_cst (ամենախիստ):

is_lock_free:

atomic-ը իսկապես lock-free է թե ներսում mutex օգտագործում (փոքր տիպեր -> lock-free):

atomic flag ինչի համար:

Stop/ready signal thread-ների միջև, առանց mutex:

Ամփոփում (Cheat Sheet)

ATOMIC = անբաժանելի operation -> race առանց mutex (փոքր operation-ի համար)

```
a++ / a.load() / a.store(x) / a.exchange(x)
a.compare_exchange_strong(exp, des) -> CAS (lock-free հիմք)
```

ATOMIC vs MUTEX:

```
atomic -> արագ (lock-free), փոքր operation (counter/flag/pointer)
mutex   -> դանդաղ (blocking), բարդ critical section
```

MEMORY ORDER: default seq_cst (խիստ) | relaxed/acquire/release (optimization)

is_lock_free() -> փոքր տիպեր սովորաբար lock-free

ԿԱՆՈՆ: counter/flag -> atomic | բարդ logic/container -> mutex